

Stack Data Structure in Java

Table of Contents

1. **Introduction to Stack**

2. Definition

3. Real-life Examples

4. Importance in Computer Science

5. **Characteristics of Stack**

6. LIFO Principle

7. Operations

8. Limitations

9. **Applications of Stack**

10. Expression Evaluation

11. Undo/Redo Functionality

12. Function Call Management

13. Balanced Parentheses Checking

14. **Implementation of Stack in Java**

15. Using Arrays

16. Using Linked List

17. Using Java Collections Framework

18. **Stack Operations in Detail**

19. Push

20. Pop

21. Peek/Top

22. isEmpty

23. isFull

24. **Time and Space Complexity**

25. Operation-wise Complexity

26. Problems and Interview Questions

27. Reverse a String using Stack

28. Balanced Brackets Problem

29. Evaluate Postfix Expression

30. Next Greater Element

31. Advanced Variations of Stack

32. Two Stacks in an Array

33. Min Stack (Get Minimum in $O(1)$)

34. Stack using Queues

35. Queue using Stacks

36. Key Takeaways

37. Conclusion

1. Introduction to Stack

A **Stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle. The last element added to the stack is the first one to be removed.

Real-life Examples

- A stack of plates: The last plate placed is the first to be removed.
- Browser history: The last visited page is the first to be revisited when pressing "back".

Importance in Computer Science

- Used in recursion, expression evaluation, parsing, and memory management.
-

2. Characteristics of Stack

- **LIFO Principle:** Last element inserted is removed first.
 - **Operations:**
 - Push (Insert)
 - Pop (Delete)
 - Peek (Access top element)
 - **Limitations:** Fixed size when implemented with arrays.
-

3. Applications of Stack

- Expression Evaluation (Postfix, Prefix)
 - Undo/Redo in text editors
 - Function call management in recursion
 - Balanced parentheses checking
-

4. Implementation of Stack in Java

a) Stack using Arrays

```
class StackArray {
    private int[] arr;
    private int top;
    private int capacity;

    StackArray(int size) {
        arr = new int[size];
        capacity = size;
        top = -1;
    }

    public void push(int x) {
        if (isFull()) {
            System.out.println("Stack Overflow");
            return;
        }
        arr[++top] = x;
    }

    public int pop() {
        if (isEmpty()) {
            System.out.println("Stack Underflow");
            return -1;
        }
        return arr[top--];
    }

    public int peek() {
        if (isEmpty()) return -1;
        return arr[top];
    }

    public boolean isEmpty() {
        return top == -1;
    }
}
```

```

    }

    public boolean isFull() {
        return top == capacity - 1;
    }
}

```

b) Stack using Linked List

```

class StackLL {
    private class Node {
        int data;
        Node next;
        Node(int data) { this.data = data; }
    }

    private Node top;

    public void push(int x) {
        Node node = new Node(x);
        node.next = top;
        top = node;
    }

    public int pop() {
        if (isEmpty()) {
            System.out.println("Stack Underflow");
            return -1;
        }
        int val = top.data;
        top = top.next;
        return val;
    }

    public int peek() {
        if (isEmpty()) return -1;
        return top.data;
    }

    public boolean isEmpty() {
        return top == null;
    }
}

```

c) Stack using Java Collections Framework

```
import java.util.Stack;

public class StackJCF {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();

        stack.push(10);
        stack.push(20);
        stack.push(30);

        System.out.println(stack.pop());
        System.out.println(stack.peek());
        System.out.println(stack.isEmpty());
    }
}
```

5. Stack Operations in Detail

- **Push:** Insert element at top.
- **Pop:** Remove element from top.
- **Peek:** View top element without removal.
- **isEmpty:** Check if stack is empty.
- **isFull:** Check if stack is full (array implementation).

6. Time and Space Complexity

Operation	Time Complexity	Space Complexity
Push	O(1)	O(1)
Pop	O(1)	O(1)
Peek	O(1)	O(1)

7. Problems and Interview Questions

a) Reverse a String using Stack

```
import java.util.Stack;

class ReverseString {
    public static String reverse(String str) {
        Stack<Character> stack = new Stack<>();
        for (char c : str.toCharArray()) stack.push(c);

        StringBuilder sb = new StringBuilder();
        while (!stack.isEmpty()) sb.append(stack.pop());
        return sb.toString();
    }
}
```

b) Balanced Brackets Problem

```
import java.util.Stack;

class BalancedParentheses {
    public static boolean isBalanced(String str) {
        Stack<Character> stack = new Stack<>();
        for (char c : str.toCharArray()) {
            if (c == '(' || c == '{' || c == '[') stack.push(c);
            else if (c == ')' && stack.peek() == '(') stack.pop();
            else if (c == '}' && stack.peek() == '{') stack.pop();
            else if (c == ']' && stack.peek() == '[') stack.pop();
            else return false;
        }
        return stack.isEmpty();
    }
}
```

c) Evaluate Postfix Expression

```
import java.util.Stack;

class PostfixEvaluation {
    public static int evaluate(String expr) {
        Stack<Integer> stack = new Stack<>();
        for (char c : expr.toCharArray()) {
            if (Character.isDigit(c)) stack.push(c - '0');
        }
    }
}
```

```

        else {
            int b = stack.pop();
            int a = stack.pop();
            switch (c) {
                case '+': stack.push(a + b); break;
                case '-': stack.push(a - b); break;
                case '*': stack.push(a * b); break;
                case '/': stack.push(a / b); break;
            }
        }
    }
    return stack.pop();
}
}

```

d) Next Greater Element

- Given an array, find the next greater element for each element using a stack.

8. Advanced Variations of Stack

- **Two Stacks in an Array:** Efficient space utilization.
- **Min Stack:** Get minimum element in O(1).
- **Stack using Queues:** Implement stack functionality using queue operations.
- **Queue using Stacks:** Implement queue functionality using stack operations.

9. Key Takeaways

- Stack is a LIFO data structure.
- Can be implemented via arrays, linked lists, or Java Collections.
- Supports constant-time operations.
- Widely used in problem-solving and system design.

10. Conclusion

Stacks are a fundamental data structure in computer science. Understanding their implementation, operations, and applications is crucial for technical interviews and real-world programming.